Tema 8

Utilización avanzada de clases

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Relaciones entre clases. Composición de clases 6
2.1 Relaciones entre clases 6
2.2 Composición de clases 9
3. Herencia. Concepto y tipos (simple y múltiple) 11
3.1 Herencia. Concepto 11
3.2 Tipos de Herencia (simple y múltiple) 13
3.2.1 Herencia Simple (Python) 13
3.2.2 Herencia Múltiple (Python) 14
3.2.3 Herencia Simple (Java) 16
3.2.4 Herencia Múltiple (Java) 17
4. Superclases y subclases 19
4.1 Clases abstractas e interfaces 20
5. Constructores y herencia 22
5.1 Constructores 22
5.2 Herencia 23
6. Modificadores en clases, atributos y métodos 25
7. Sobreescritura de métodos 29
8. Clases y métodos abstractos y finales 31
8.1 Métodos abstractos 32
8.2 Métodos finales 33
9. Interfaces. Clases abstractas vs interface 35
9.1 Clases abstractas 35
10. Interfaces. Diferencia clases abstractas vs interfaces 37
10.1 Diferencia entre las clases abstractas y las interfaces 37
11. Polimorfismo 41
11.1 Concepto 41

12. Polimorfismo en tiempo de compilación (sobrecarga) y polimorfismo en tiempo de ejecución (ligadura dinámica) 43
12.1 Polimorfismo: Comprobación estática y dinámica de tipos 45
13. Conversiones de tipos entre objetos (casting) 48
13.1 Upcasting 50
13.2 Downcasting 51
13.3. Diferencia entre casting implícito y el casting explícito vs Upcasting y Downcasting ... 52
14. Clases y tipos genéricos o parametrizados 54
14.1 Introducción a las clases y tipos genéricos o parametrizados 54
14.2 ¿Qué son las clases genéricas? 54
14.3 ¿Qué son los métodos genéricos? 55
14.4 ¿Qué son los tipos comodín? 55
15. Resumen 56
Glosario de términos 57
Bibliografía 58

Tech Hub <FP> euroformac

Objetivos de la unidad

Los objetivos de esta unidad son:

- Conocer los conceptos avanzados de las clases en programación.
- Aprender a utilizar constructores y herencia para crear jerarquías de clases más complejas.
- Comprender la importancia de los modificadores de acceso y su aplicación en atributos y métodos de las clases.
- Conocer la aplicación de la sobreescritura de métodos para alterar el funcionamiento de los métodos heredados.
- Conocer las clases y métodos abstractos y finales y su uso en el diseño de clases.
- Aprender el concepto de interfaces y su diferencia con las clases abstractas.
- Comprender el concepto de polimorfismo y cómo se puede aplicar en la programación orientada a objetos.
- Saber la diferencia entre el polimorfismo en tiempo de compilación y en tiempo de ejecución.
- Comprender la comprobación estática y dinámica de tipos en polimorfismo.
- Aprender sobre las conversiones de tipos entre objetos y su aplicación en la programación.
- Conocer las clases y tipos genéricos o parametrizados y su uso en la programación orientada a objetos.

1. Introducción

La utilización avanzada de clases es un tema crucial en programación orientada a objetos. Esta unidad se enfoca en profundizar en el uso de clases y sus principales características, como la herencia, polimorfismo, encapsulamiento y abstracción.

Además, abordaremos temas avanzados como el manejo de excepciones, la sobrecarga de operadores y la implementación de métodos estáticos y métodos de clase.

En resumen, este módulo nos permitirá profundizar en las capacidades de las clases y cómo podemos utilizarlas para crear soluciones de programación más efectivas y escalables.

2. Relaciones entre clases. Composición de clases

En la programación orientada a objetos, las relaciones entre clases son un aspecto crucial. En este apartado, abordaremos la técnica de composición de clases, la cual nos permite crear objetos más elaborados al combinar múltiples objetos más simples.

La **composición de clases** se basa en el concepto de que un objeto puede contener otros objetos como parte de su estructura interna. De esta manera, podemos construir objetos más complejos a partir de objetos más simples, cada uno con su propio conjunto de propiedades y métodos.

En este epígrafe, exploraremos cómo podemos **crear relaciones entre clases mediante la composición**. Veremos cómo podemos **diseñar clases que contengan otras clases como parte de su estructura, y cómo podemos acceder a los miembros de las clases internas desde la clase contenedora.** Además, discutiremos algunos casos de uso comunes de la composición de clases, como la creación de objetos complejos que representan entidades del mundo real, como un coche que contiene un motor, una transmisión y otros componentes.
En resumen, este epígrafe es fundamental para comprender cómo las clases pueden relacionarse entre sí de manera efectiva y cómo podemos utilizar la composición para construir objetos más complejos.

2.1 Relaciones entre clases

En la programación orientada a objetos, las **clases** son como plantillas que nos permiten crear objetos que tienen un conjunto de propiedades y métodos en común. Estos objetos pueden interactuar entre sí, y a menudo necesitamos establecer relaciones entre ellos.

Las **relaciones entre clases** se refieren a cómo las clases están relacionadas entre sí en un programa.

Hay diferentes tipos de relaciones, como la **herencia**, la **agregación** y la **composición**.
En el caso de la **composición de clases** nos enfocamos en cómo podemos construir objetos complejos combinando varios objetos más simples. Es decir, una clase puede estar compuesta de otras clases como parte de su estructura interna. Cada clase interna tiene su propio conjunto de

propiedades y métodos, y puede ser utilizada como una parte del objeto de la clase contenedora.

Por ejemplo, imagina que quieres crear un objeto que represente un coche. Este coche tendría diferentes partes, como un motor, una transmisión, ruedas y otros componentes. En lugar de crear un objeto gigante para representar el coche completo, puedes crear objetos separados para cada parte del coche y luego combinarlos en un objeto coche más complejo utilizando la composición.

De esta manera, el objeto coche está compuesto de otros objetos más simples, como el objeto motor, el objeto transmisión, el objeto ruedas, etc. Cada objeto interno tiene su propia funcionalidad y propiedades, y puede ser manipulado por la clase contenedora del objeto coche.

Para seguir con el ejemplo del coche, podríamos tener una **clase Motor** que represente el motor del coche, con sus propias propiedades y métodos. Por ejemplo:

```
class Motor:

    def __init__(self, cilindros, combustible):

        self.cilindros = cilindros

        self.combustible = combustible

        self.encendido = False


    def encender(self):

        self.encendido = True

        print("Motor encendido.")


    def apagar(self):

        self.encendido = False

        print("Motor apagado.")
```

Luego, podríamos tener una **clase Transmisión** que represente la transmisión del coche:

```
class Transmision:
    def __init__(self, tipo):
        self.tipo = tipo
        self.marcha = 0

    def cambiar_marcha(self, nueva_marcha):
        self.marcha = nueva_marcha
        print(f"Transmisión cambiada a marcha {nueva_marcha}.")
```

Finalmente, podemos combinar estas clases en una **clase Coche** utilizando la composición. La clase Coche tendría instancias de las clases Motor y Transmisión como propiedades, y podría utilizar sus métodos para encender el motor, cambiar de marcha, etc.

```
class Coche:
    def __init__(self, motor, transmision):
        self.motor = motor
        self.transmision = transmision

    def encender(self):
        self.motor.encender()

    def apagar(self):
        self.motor.apagar()

    def cambiar_marcha(self, nueva_marcha):
        self.transmision.cambiar_marcha(nueva_marcha)
```

Luego, podemos crear una **instancia de la clase Coche** utilizando instancias de las clases Motor y Transmisión como argumentos:

```
motor = Motor(6, "gasolina")

transmision = Transmision("manual")

coche = Coche(motor, transmision)
```

Ahora, podemos manipular el objeto coche utilizando sus métodos, que a su vez utilizan los métodos de las instancias de las clases Motor y Transmisión.

```
coche.encender()   # "Motor encendido."

coche.cambiar_marcha(1)    # "Transmisión cambiada a marcha 1."

coche.cambiar_marcha(2)    # "Transmisión cambiada a marcha 2."

coche.apagar()    # "Motor apagado."
```

Como puedes ver, la composición de clases nos permite crear objetos más complejos a partir de objetos más simples, y nos permite utilizar las funcionalidades de cada objeto interno en la clase contenedora. Esto puede hacer que nuestros programas sean más modulares y fáciles de mantener y extender en el futuro.

2.2 Composición de clases

La composición de clases es un concepto clave en la **programación orientada a objetos** (**POO**) que se utiliza para crear objetos complejos combinando objetos más simples.

En la composición de clases, un objeto se compone de uno o más objetos más pequeños, cada uno de los cuales representa una parte o un aspecto específico del objeto más grande. Por ejemplo, un objeto "Coche" puede estar compuesto de objetos más pequeños, como un objeto "Motor", un objeto "Ruedas", un objeto "Carrocería", etc.

Estos objetos más pequeños se conocen como **objetos compuestos o componentes**, y el objeto más grande se conoce como **objeto compuesto o contenedor**. En POO, la composición se puede utilizar para modelar objetos complejos de una manera más sencilla y modular.

Un ejemplo de composición de clases en la vida real podría ser una bicicleta. Una bicicleta está compuesta de varias partes, como el cuadro, las ruedas, el manillar, el sillín, etc. Cada una de estas partes es un objeto separado con sus propias propiedades y comportamientos. Cuando se combinan estas partes en un objeto "Bicicleta", se crea un objeto más complejo y funcional.

En POO, la composición se puede representar mediante una clase que tiene objetos de otras clases como propiedades. Por ejemplo, en la clase "Coche" se podrían definir las propiedades "motor", "ruedas" y "carrocería" como objetos de otras clases correspondientes. La clase "Coche" podría luego utilizar los métodos y propiedades de estos objetos para construir un objeto completo y funcional.

Importante

Es importante comprender la composición de clases ya que puede ayudar a crear programas más estructurados y fáciles de mantener.

3. Herencia. Concepto y tipos (simple y múltiple)

La **herencia** es un concepto clave en la **programación orientada a objetos** (**POO**) que permite a una clase heredar propiedades y métodos de otra clase.

En POO, las clases se organizan en una jerarquía, donde las clases superiores se conocen como **clases base o clases padre**, y las clases inferiores se conocen como **clases hijas o clases derivadas**. La herencia permite a las clases hijas heredar las propiedades y los métodos de sus clases padre, lo que les permite extender y personalizar el comportamiento de la clase base.

La herencia se clasifica generalmente en dos tipos: **herencia simple** y **herencia múltiple**. La herencia simple se refiere a una clase derivada que hereda propiedades y métodos de una única clase base. Por ejemplo, una clase "Coche" podría heredar propiedades y métodos de una clase "Vehículo". La herencia múltiple, por otro lado, se refiere a una clase derivada que hereda propiedades y métodos de más de una clase base. Por ejemplo, una clase "Coche" podría heredar propiedades y métodos de una clase "Vehículo" y una clase "Motor".

Importante

Es importante entender la herencia en POO, ya que permite la reutilización de código y la creación de programas más eficientes y organizados.

3.1 Herencia. Concepto

La POO emplea el concepto de herencia para generar nuevas clases mediante la utilización de clases previamente definidas. Esta técnica resulta esencial en la programación orientada a objetos. En POO una clase se puede considerar como una plantilla o un molde para crear objetos. Una clase define un conjunto de propiedades y métodos que un objeto de esa clase puede tener. La **herencia permite a una clase hija heredar todas las propiedades y métodos de su clase padre o clases padres**.

Por ejemplo, considera la clase "Animal". Esta clase podría tener propiedades como "nombre", "edad", "especie", y métodos como "alimentar" o "dormir". Ahora, imagino que vamos a crear una clase "Perro". En lugar de escribir toda la clase "Perro" desde cero, podemos simplemente heredar todas las

propiedades y métodos de la clase "Animal", ya que un perro es también un tipo de animal.

En POO, para heredar una clase, simplemente se define la nueva clase utilizando la palabra clave "extends" seguida del nombre de la clase padre. Por ejemplo:

```
class Animal {
    String nombre;
    int edad;
    String especie;

    void alimentar() {
      // código para alimentar al animal
    }

    void dormir() {
      // código para hacer dormir al animal
    }
  }

  class Perro extends Animal {
    // todas las propiedades y métodos de Animal se heredan automáticamente
    // podemos añadir propiedades y métodos adicionales específicos para la clase Perro si
lo necesitamos
    String raza;

    void ladrar() {
      // código para hacer que el perro ladre
    }
  }
```

En este ejemplo, la clase "Perro" hereda todas las propiedades y métodos de la clase "Animal" , e incluso una propiedad adicional "raza" y un método "ladrar".

La herencia es importante en POO porque permite la reutilización de código y la creación de jerarquías de clases que reflejen la relación de "es un tipo de" entre clases. En este ejemplo, podemos decir que "un perro es un tipo de animal". Al heredar la clase "Animal", la clase "Perro" puede utilizar todos los métodos y propiedades de la clase "Animal", lo que puede ahorrar tiempo y esfuerzo en la creación de la clase "Perro" desde cero.

Importante

Es importante tener en cuenta que la herencia permite a una clase hija heredar todas las propiedades y métodos de su clase padre o clases padres. Esto significa que podemos crear jerarquías de clases que reflejen la relación de "es un tipo de" entre las clases, lo que puede ahorrar tiempo y esfuerzo en la creación de nuevas clases desde cero.

3.2 Tipos de Herencia (simple y múltiple)

Veamos los tipos de herencia en Python y Java con algunos ejemplos:

3.2.1 Herencia Simple (Python)

La **herencia simple en Python** se define utilizando la siguiente sintaxis:

```
# Definición de una herencia simple en Python
class ClaseHija(ClasePadre):
    def __init__(self, atributos_de_la_hija):
        super().__init__(atributos_de_la_padre)
        # otros atributos y métodos de la hija
```

Veamos un ejemplo:

```
# Ejemplo de una herencia simple en Python
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hablar(self):
        print("El animal está haciendo un sonido.")

class Perro(Animal):
    def __init__(self, nombre):
        super().__init__(nombre)

    def hablar(self):
        print("El perro hace 'guau-guau'")
```

En este ejemplo, la clase Perro hereda de la clase Animal y sobrescribe el método hablar para imprimir un mensaje específico. Al crear una instancia de la clase Perro, se le debe pasar un argumento para el nombre del perro:

```
mi_perro = Perro("Bobby")
print(mi_perro.nombre)
# Salida: Bobby
mi_perro.hablar()
# Salida: El perro hace 'guau-guau'
```

3.2.2 Herencia Múltiple (Python)

La herencia múltiple en Python se define utilizando la siguiente sintaxis:

```
# Definición de una herencia multiple en Python
class ClaseHija(ClasePadre1, ClasePadre2, ...):
    def __init__(self, atributos_de_la_hija):
```

```
        super().__init__(atributos_de_la_padre1, atributos_de_la_padre2, ...)
        # otros atributos y métodos de la hija
```

Veamos un ejemplo:

```
# Ejemplo de una herencia multiple en Python
class A:
    def hacer_algo(self):
        print("Clase A")

class B:
    def hacer_algo(self):
        print("Clase B")

class C(A, B):
    pass

objeto_c = C()
objeto_c.hacer_algo()
# Salida: Clase A
```

En este ejemplo, la clase C hereda de las clases A y B, las cuales tienen un método con el mismo nombre hacer_algo. Al llamar al método hacer_algo del objeto de la clase C, se ejecuta el método de la clase A, ya que se especificó que A es la primera clase padre en la definición de la clase C.

Importante

Es importante tener en cuenta que, al igual que en el caso de Java, la herencia múltiple puede ser compleja y debe usarse con precaución para evitar problemas de ambigüedad y complejidad en el código.

3.2.3 Herencia Simple (Java)

A continuación, se explican los tipos de herencia en Java con ejemplos:

La **herencia simple** es cuando una subclase extiende directamente de una única superclase. En Java, la herencia simple se puede lograr mediante la palabra clave "extends". Ejemplo:

```
// Ejemplo de herencia simple
  public class Animal {
    String name;
    int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void speak() {
        System.out.println("El animal hace un sonido.");
    }
}

public class Dog extends Animal {
    String breed;

    public Dog(String name, int age, String breed) {
        super(name, age);
        this.breed = breed;
    }

    public void fetch() {
        System.out.println("El perro busca y trae el objeto.");
```

En el ejemplo anterior, la clase "Dog" extiende la clase "Animal". La clase "Dog" es la subclase y la clase "Animal" es la superclase. La subclase "Dog" hereda todos los campos y métodos públicos de la superclase "Animal". Además, también tiene un método "fetch" adicional que no está presente en la superclase "Animal".

3.2.4 Herencia Múltiple (Java)

La **herencia múltiple** es cuando una subclase extiende de varias superclases. Java no admite la herencia múltiple de clases, pero sí admite la herencia múltiple de interfaces. En Java, la herencia múltiple se puede lograr mediante la palabra clave "implements".

Ejemplo:

```
// Ejemplo de herencia multiple
public interface CanSwim {
    void swim();
}

public interface CanFly {
    void fly();
}

public class Duck implements CanSwim, CanFly {
    @Override
    public void swim() {
        System.out.println("El pato nada en el agua.");
    }

    @Override
    public void fly() {
        System.out.println("El pato vuela en el cielo.");
    }
```

En el ejemplo anterior, la clase "Duck" implementa las interfaces "CanSwim" y "CanFly". La clase "Duck" hereda los métodos definidos en ambas interfaces y los implementa en su propia clase. La herencia múltiple de interfaces permite que una clase pueda tener varios comportamientos sin necesidad de heredar de múltiples clases.

4. Superclases y subclases

En programación, las **superclases** y **subclases** son términos utilizados en el contexto de la herencia de clases. La herencia es un mecanismo en el que una clase puede adquirir propiedades y comportamientos de otra clase, llamada superclase. La clase que hereda de la superclase se llama **subclase**.

Por ejemplo, si tenemos una superclase "Animal" con propiedades y comportamientos generales, podemos crear una subclase "Perro" que herede esas propiedades y comportamientos y añada sus propias propiedades y comportamientos específicos de los perros.

En Java, para definir una subclase se utiliza la palabra clave "extends" seguida del nombre de la superclase. Por ejemplo:

```
// Ejemplo de definir una subclase en Java
public class Perro extends Animal {
    // propiedades y métodos específicos de los perros
}
```

En Python, para definir una subclase, se indica entre paréntesis el nombre de la superclase después del nombre de la subclase. Por ejemplo:

```
# Ejemplo de definir una subclase en Python
class Perro(Animal):
    # propiedades y métodos específicos de los perros
```

Importante

Es importante tener en cuenta que las subclases heredan las propiedades y métodos de su superclase, pero también pueden modificar o añadir nuevos comportamientos y propiedades propias. Además, una superclase puede tener varias subclases, lo que nos permite crear una jerarquía de clases cada vez más especializadas.

4.1 Clases abstractas e interfaces

Clases abstractas e interfaces también son importantes en la relación de superclase y subclase. Una clase abstracta es una clase que no se puede instanciar, sino que se utiliza como modelo para otras clases. Cuando una clase hereda de una clase abstracta, debe proporcionar implementaciones concretas para todos los métodos abstractos definidos en la superclase.

Por otra parte, en programación se entiende por interfaz a un conjunto de métodos abstractos que están disponibles para ser implementados por una clase.

Las interfaces permiten definir un conjunto de métodos que las clases deben implementar sin definir su implementación. Una clase tiene la capacidad de implementar múltiples interfaces. Esto implica que la clase puede heredar comportamientos y funcionalidades de diversas fuentes.

Por ejemplo, en Java, podemos definir una clase abstracta "Figura" con un método abstracto "calcularArea", que cada subclase debe implementar. Además, podemos definir una interfaz "Dibujable" con un método "dibujar". Luego, podemos tener una subclase "Cuadrado" que hereda de "Figura" y que implementa "Dibujable":

```
// Ejemplo de definir una clase abstracta en Java
public abstract class Figura {
    public abstract double calcularArea();
}

public interface Dibujable {
    void dibujar();
}

public class Cuadrado extends Figura implements Dibujable {
    private double lado;

    public Cuadrado(double lado) {
        this.lado = lado;
    }
```

```
    @Override
    public double calcularArea() {
        return lado * lado;
    }

    @Override
    public void dibujar() {
        // Código para dibujar un cuadrado
    }
}
```

En resumen, las superclases y subclases son una parte fundamental de la programación orientada a objetos y permiten crear jerarquías de clases que heredan propiedades y comportamientos de una clase base. Las clases abstractas e interfaces son herramientas adicionales para definir comportamientos y propiedades comunes en las clases y permitir la creación de objetos polimórficos.

Más info

Si deseas ampliar tu conocimiento sobre la herencia en POO, te recomendamos la visualización de este vídeo sobre la herencia:

https://www.youtube.com/watch?v=wqoyQ3BxK4A&ab_channel=pildorasinformaticas

5. Constructores y herencia

5.1 Constructores

En este contexto, la **utilización avanzada de clases** se refiere a técnicas más avanzadas para crear y manejar objetos y clases en POO. Uno de los aspectos más importantes de la utilización avanzada de clases es el uso de **constructores** y **herencia**.

En programación, un **constructor** es una función especial cuya finalidad es la de inicializar los objetos durante su creación. Los constructores pueden tomar argumentos y asignar valores a las propiedades de un objeto. En POO, una subclase hereda todos los miembros de su superclase, incluyendo su constructor. Por lo tanto, una subclase puede llamar al constructor de su superclase para inicializar las propiedades que hereda.

La **herencia** es una técnica que permite definir una nueva clase basada en una clase existente. En la programación orientada a objetos, la clase resultante de la creación de una nueva a partir de una ya existente es conocida como subclase o clase derivada, mientras que la clase previamente existente se conoce como superclase o clase base. La subclase hereda todas las propiedades y métodos de la superclase, y puede agregar nuevas propiedades y métodos, o modificar los existentes.

Más info

Aquí podrás ver un tutorial de Real Python sobre herencia en Python: https://realpython.com/inheritance-composition-python/

Por ejemplo, en Java, podemos tener una clase "Vehiculo" con propiedades como "marca", "modelo" y "anio". Luego, podemos tener una subclase "Automovil" que hereda de "Vehiculo" y agrega nuevas propiedades como "numeroDePuertas" y "tipoDeTransmision". En la subclase, podemos llamar al constructor de la superclase para inicializar las propiedades que hereda:

```
// Ejemplo en Java:

public class Vehiculo {

    private String marca;

    private String modelo;

    private int anio;
```

```
    public Vehiculo(String marca, String modelo, int anio) {
        this.marca = marca;
        this.modelo = modelo;
        this.anio = anio;
    }
}

public class Automovil extends Vehiculo {
    private int numeroDePuertas;
    private String tipoDeTransmision;

    public Automovil(String marca, String modelo, int anio, int numeroDePuertas,
String tipoDeTransmision) {
        super(marca, modelo, anio);
        this.numeroDePuertas = numeroDePuertas;
        this.tipoDeTransmision = tipoDeTransmision;
    }
}
```

En resumen, la utilización avanzada de clases en POO implica el uso de constructores y herencia para crear objetos y clases más complejas y reutilizables. Los constructores se utilizan para inicializar objetos cuando se crean, y la herencia permite definir nuevas clases basadas en clases existentes y heredar propiedades y métodos de ellas.

5.2 Herencia

Dentro de la programación orientada a objetos, la herencia es una característica fundamental que posibilita que las clases adquieran atributos y métodos de otras clases. Por otra parte, la utilización de constructores en estas clases permite crear objetos de forma más eficiente.

En la utilización avanzada de clases es común que necesitemos definir constructores en nuestras clases hijas para poder inicializar los atributos heredados de la clase padre. En Java podemos hacer uso del constructor de la clase padre mediante el uso de la palabra clave super.

Por ejemplo, supongamos que tenemos una clase Vehiculo que tiene un atributo color y un método arrancar(). Ahora, podemos crear una clase Coche que herede de Vehiculo y tenga su propio atributo numPuertas. En este caso, definiríamos un constructor en Coche que llame al constructor de Vehiculo usando super(color) para inicializar el atributo color heredado, y luego inicializaríamos el atributo numPuertas.

En Python podemos hacer uso del método super() para llamar al constructor de la clase padre y pasarle los argumentos necesarios. Por ejemplo, si tenemos una clase Animal con un atributo nombre, y queremos crear una clase Gato que herede de Animal y tenga su propio atributo maullido, podríamos definir el constructor de Gato de la siguiente manera:

```
# Ejemplo en python
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

class Gato(Animal):
    def __init__(self, nombre, maullido):
        super().__init__(nombre)
        self.maullido = maullido
```

En resumen, la utilización avanzada de clases en programación nos permite hacer uso de la herencia y los constructores para crear código más eficiente y reutilizable.

Más info

En enlace podrás obtener información complementaria a lo estudiado en el temario encontrada en la página de Microsoft: https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/object-oriented/inheritance

6. Modificadores en clases, atributos y métodos

La utilización de **modificadores** en programación es una herramienta importante para controlar la visibilidad y acceso a los atributos y métodos de una clase. En esta sección, se explorará cómo los modificadores pueden ser utilizados en clases, atributos y métodos para mejorar la seguridad y el encapsulamiento del código.

En general hay cuatro tipos de modificadores en programación: **public**, **private**, **protected** y **default** (también conocido como ***package-private***). Cada uno de estos modificadores controla el nivel de acceso a un atributo o método en una clase.

- El modificador "public" permite que un atributo o método sea accedido desde cualquier otra clase, incluso fuera del paquete actual.
  Por ejemplo:

```
public class Persona {
    public String nombre;

    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
 }
```

En este ejemplo, la clase "Persona" tiene un atributo "nombre" y un método "saludar", ambos marcados como "public". Esto significa que cualquier otra clase puede acceder a ellos.

- El modificador "private" limita el acceso a un atributo o método sólo dentro de la clase en la que se define. Por ejemplo:

```
public class Persona {
    private String nombre;

    private void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
```

En este ejemplo, la clase "Persona" tiene un atributo "nombre" y un método "saludar", ambos marcados como "private". Esto significa que sólo la propia clase "Persona" puede acceder a ellos.

- El modificador "protected" permite que un atributo o método sea accedido dentro de la clase en la que se define y también en cualquier subclase que herede de esa clase. No se pueden acceder desde otras clases ni desde fuera de la clase.
  Por ejemplo:

```
public class Animal {
    protected String tipo;

    protected void emitirSonido() {
        System.out.println("El " + tipo + " hace algún sonido");
    }
}

public class Perro extends Animal {
    public Perro() {
        tipo = "perro";
    }

    public void ladrar() {
        System.out.println("Guau guau!");
        emitirSonido();
    }
}
```

En este ejemplo, la clase "Animal" tiene un atributo "tipo" y un método "emitirSonido", ambos marcados como "protected". Esto significa que la clase "Perro", que hereda de "Animal", también puede acceder a ellos.

- El modificador "default" (o "*package-private*") limita el acceso a un atributo o método sólo dentro del mismo paquete. Si no se especifica ningún modificador, el atributo o método tendrá el modificador "default". Por ejemplo:

También conocido como "**amistoso**" o "**sin modificador**", los miembros de una clase con este modificador son accesibles desde la propia clase y desde otras clases que estén en el mismo paquete.

```
class Cuenta {
    double saldo;

    void depositar(double cantidad) {
        saldo += cantidad;
    }
}
```

En este ejemplo, la clase "Cuenta" tiene un atributo "saldo" y un método "depositar", ambos sin especificar ningún modificador. Esto significa que sólo las clases dentro del mismo paquete que "Cuenta" pueden acceder a ellos.

La utilización adecuada de modificadores es esencial para asegurar que los atributos y métodos sean accesibles sólo donde sea necesario y para proteger la integridad del código.

Los **modificadores en atributos y métodos** son los mismos que en clases. Además, también se pueden utilizar los siguientes:

- **static**: Esta palabra clave se utiliza para declarar atributos y métodos que pertenecen a la clase en sí misma, en lugar de a una instancia de la clase.
- **final**: Esta palabra clave se utiliza para declarar que un atributo o método no puede ser modificado después de su inicialización o definición.
- **abstract**: Esta palabra clave se utiliza para declarar que una clase o método no tiene una implementación completa y debe ser completado en una subclase.
- **synchronized**: Esta palabra clave se utiliza para declarar que un método solo puede ser accedido por un hilo a la vez.
- **volatile**: Esta palabra clave se utiliza para declarar que un atributo puede ser modificado por varios hilos al mismo tiempo.

Estos modificadores pueden ser combinados para crear diferentes niveles de visibilidad y control de acceso en las clases, atributos y métodos. Es importante tener en cuenta que el uso adecuado de los modificadores es

fundamental para la buena práctica de programación, así como también que losmodificadores de acceso también se aplican a los constructores de la clase.

```
public class Ejemplo {
    private int atributoPrivado;
    public Ejemplo() {
        atributoPrivado = 0;
    }
}
```

En este ejemplo, el constructor de la clase Ejemplo tiene el modificador public, lo que significa que puede ser accedido desde cualquier parte del programa. Además, dentro del constructor se accede al atributo privado de la clase para inicializarlo con el valor 0.
Es fundamental comprender y aplicar correctamente los modificadores de acceso para evitar errores y mantener la integridad del código en nuestro programa.

7. Sobreescritura de métodos

La **sobreescritura de métodos** es una técnica que permite a las subclases redefinir un método ya definido en la superclase. Esto significa que la subclase proporciona una nueva implementación para el método que anula la implementación de la superclase.

Supongamos que tenemos una clase Animal con un método hacerSonido(), que simplemente imprime un sonido genérico. Queremos crear una subclase Perro que haga un ladrido específico cuando se llama al método hacerSonido(). Para hacerlo, debemos utilizar la sobreescritura de métodos.

En Java, la sobreescritura de métodos se realiza de la siguiente manera:

```
public class Animal {
    public void hacerSonido() {
        System.out.println("Haciendo un sonido genérico.");
    }
}

public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Guau!");
    }
}
```

En el ejemplo anterior, la subclase Perro hereda el método hacerSonido() de la superclase Animal, pero lo sobrescribe con su propia implementación. La anotación @Override es opcional, pero se utiliza para indicar explícitamente que se está sobrescribiendo un método de la superclase.

Para probarlo, podemos crear una instancia de la clase Perro y llamar al método hacerSonido():

```
public class Test {
    public static void main(String[] args) {
```

```
        Perro miPerro = new Perro();
        miPerro.hacerSonido(); // Imprime: ¡Guau!
    }
}
```

Al ejecutar el programa, se imprimirá ¡ Guau!, que es la implementación del método hacerSonido() en la subclase Perro.

Importante

Es importante destacar que la sobreescritura de métodos solo se puede realizar en métodos públicos o protegidos de la superclase, y la subclase no puede reducir la visibilidad de un método que sobrescribe.

8. Clases y métodos abstractos y finales

En programación es común utilizar clases como plantillas para objetos y estructuras de datos. Sin embargo, hay ocasiones en las que necesitamos declarar una clase como incompleta o no instanciable. Para esto se utilizan las **clases abstractas**. Por otro lado, existen situaciones en las que no queremos que una clase o método sea sobrescrito o modificado. Para ello, se utilizan las clases y métodos finales.

En este epígrafe, aprenderás sobre las clases y métodos abstractos y finales.

- **Clases Abstractas**: En programación, se conoce como clases abstractas a aquellas que no pueden ser instanciadas, lo que significa que no es posible crear objetos a partir de ellas. Sin embargo, se pueden utilizar como base para la creación de otras clases que sí permitan la creación de objetos. Para definir una clase como abstracta, se utiliza la palabra clave "abstract" antes de la declaración de la clase.

Un ejemplo de una clase abstracta en Java sería el siguiente:

```
abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public abstract void makeSound();
 }
```

En este ejemplo, la clase "Animal" es abstracta, lo que significa que no se pueden crear objetos a partir de ella. En su lugar, se utilizará como base para otras clases que sí se puedan instanciar. La clase "Animal" tiene un constructor que inicializa el nombre del animal y un método abstracto "makeSound()" que será implementado por las clases que extiendan esta clase abstracta.

8.1 Métodos abstractos

Un método abstracto en programación es aquel que no cuenta con una implementación definida en la clase abstracta, es decir, carece de un cuerpo de código. Por tanto, se espera que las subclases de la clase abstracta definan la implementación de este método en particular.

Un ejemplo de un método abstracto en Java sería el siguiente:

```
abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public abstract void makeSound();
 }

 class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    public void makeSound() {
        System.out.println("Woof!");
    }
 }
```

En este ejemplo, la clase "Animal" es abstracta y tiene un método abstracto "makeSound()". La clase "Dog" extiende la clase "Animal" e implementa el método "makeSound()" con una implementación concreta.

- **Clases Finales:** Una clase final es una clase que no se puede extender, es decir, no se pueden crear subclases a partir de ella. Las clases finales

se definen mediante la palabra clave "final" antes de la declaración de la clase.

Un ejemplo de una clase final en Java sería el siguiente:

```
final class Circle {
    private final double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }
}
```

En este ejemplo, la clase "Circle" es final, lo que significa que no se pueden crear subclases a partir de ella. Además, la variable "radius" es final, lo que significa que su valor no puede ser cambiado después de su inicialización en el constructor.

8.2 Métodos finales

Métodos finales son métodos que no pueden ser sobrescritos por una subclase. Esto significa que el comportamiento del método es inmutable y no puede ser cambiado por una subclase. Para declarar un método como final, se usa el modificador final.

Veamos un ejemplo en Java:

```
public class Figura {
    // Atributos
    protected int base;
    protected int altura;
```

```
    // Constructor
    public Figura(int base, int altura) {
        this.base = base;
        this.altura = altura;
    }

    // Método final
    public final void mostrarArea() {
        int area = (this.base * this.altura) / 2;
        System.out.println("El área de la figura es: " + area);
    }
}

public class Triangulo extends Figura {
    // Constructor
    public Triangulo(int base, int altura) {
        super(base, altura);
    }

    // Intento de sobrescribir el método final
    public void mostrarArea() {
        System.out.println("El área del triángulo es: " + (this.base *
this.altura) / 2);
    }
}
```

En este ejemplo, tenemos una clase Figura que tiene un método mostrarArea() declarado como final. La clase Triangulo hereda de Figura y trata de sobrescribir el método mostrarArea(). Sin embargo, esto no es posible porque el método es final en la clase padre.

9. Interfaces. Clases abstractas vs interface

Las **interfaces y las clases abstractas** son dos herramientas importantes en la programación orientada a objetos. Ambas permiten definir métodos que deben ser implementados por las clases que las utilizan. Sin embargo, existen algunas diferencias importantes entre ellas que es importante conocer para saber cuál utilizar en cada situación.

9.1 Clases abstractas

Una **clase abstracta** es una clase que no puede ser instanciada directamente. Es decir, no se puede crear un objeto a partir de ella. En su lugar, se utilizan para definir una estructura común para un grupo de clases relacionadas. Las clases que extienden una clase abstracta deben implementar todos sus métodos abstractos, es decir, aquellos que han sido definidos, pero no implementados en la clase abstracta.

Veamos un ejemplo de una clase abstracta en Java:

```
public abstract class Animal {
    public abstract void hacerSonido();
    public void dormir() {
        System.out.println("Zzzzz");
    }
}
```

En este ejemplo, la clase Animal es abstracta porque tiene al menos un método abstracto (hacerSonido()) que no ha sido implementado. También tiene un método concreto (dormir()) que ha sido implementado y que puede ser utilizado por las clases que extienden esta clase abstracta.

Ahora supongamos que queremos crear una clase Perro que extienda de Animal. Como Animal es una clase abstracta, no podemos crear un objeto directamente de ella. En su lugar, tendremos que crear una subclase que implemente el método abstracto hacerSonido():

```
public class Perro extends Animal {
    public void hacerSonido() {
```

```
        System.out.println("Guau");
    }
}
```

En este ejemplo, la clase Perro implementa el método abstracto hacerSonido() definido en la clase abstracta Animal.

10. Interfaces. Diferencia clases abstractas vs interfaces

Las interfaces comparten similitudes con las clases abstractas, ya que ambas establecen una estructura común para un conjunto de clases relacionadas. No obstante, se diferencian en que **las interfaces no pueden contener implementaciones de métodos**. Todas las **variables** en una interfaz deben ser **public**, **static** y **final**, y todos los **métodos** deben ser **public** y **abstractos**.

Veamos un ejemplo de una interfaz en Java:

```
public interface Vehiculo {
    public void acelerar();
    public void frenar();
}
```

En este ejemplo, la interfaz Vehiculo define dos métodos que deben ser implementados por cualquier clase que la utilice: acelerar() y frenar().

Ahora supongamos que queremos crear una clase Coche que implemente la interfaz Vehiculo. Para hacerlo, debemos implementar todos los métodos definidos en la interfaz:

```
public class Coche implements Vehiculo {
    public void acelerar() {
        System.out.println("El coche está acelerando.");
    }
    public void frenar() {
        System.out.println("El coche está frenando.");
    }
}
```

10.1 Diferencia entre las clases abstractas y las interfaces

Las **clases abstractas** pueden tener tanto métodos abstractos como métodos concretos. Es decir, métodos que ya tienen una implementación definida.

Además, las clases abstractas pueden tener constructores y atributos como cualquier otra clase.

En programación, las interfaces establecen un conjunto de reglas que una clase debe seguir. En una interfaz, solo se pueden declarar métodos y constantes, pero no se pueden implementar. Cualquier clase que implemente una interfaz debe proporcionar una implementación para todos los métodos que se han declarado en la interfaz.

La distinción principal entre las clases abstractas y las interfaces reside en que una clase solo puede heredar de una clase abstracta, pero puede implementar varias interfaces. Además, una clase abstracta puede contar con métodos tanto abstractos como concretos, mientras que una interfaz únicamente puede tener métodos abstractos y constantes.

Aquí se aprecian algunos ejemplos en Java:

```
// Ejemplo de clase abstracta
public abstract class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }

    public abstract void makeSound(); // método abstracto que las subclases
deben implementar
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name);
```

```
    }

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }
}

// Ejemplo de interfaz
public interface Drawable {
    void draw();
}

public class Circle implements Drawable {
    private int x;
    private int y;
    private int radius;

    public Circle(int x, int y, int radius) {
        this.x = x;
        this.y = y;
        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing circle at (" + x + "," + y + ") with radius
" + radius);
    }
}
```

En este ejemplo, Animal es una clase abstracta que tiene un método abstracto makeSound(). La clase Dog hereda de Animal y proporciona una implementación para el método makeSound().

La interfaz Drawable tiene un método draw(). La clase Circle implementa la interfaz Drawable y proporciona una implementación para el método draw().

Tech Hub <FP> euroformac

11. Polimorfismo

El **polimorfismo** es un concepto muy importante en la programación orientada a objetos que permite que los objetos de diferentes clases se comporten de manera similar en determinadas situaciones. Este concepto se basa en la capacidad de los objetos de una clase para tomar diferentes formas o comportarse de diferentes maneras según las circunstancias.

11.1 Concepto

El polimorfismo se puede lograr a través de dos mecanismos: la **sobrecarga de métodos** y la **sobreescritura de métodos**. La sobrecarga de métodos en programación se refiere a la práctica de definir múltiples métodos en una misma clase que comparten el mismo nombre pero diferentes parámetros. Por otro lado, la sobreescritura de métodos se refiere a la capacidad de una subclase de proveer su propia implementación de un método heredado de su superclase.

El polimorfismo se puede entender mejor a través de un ejemplo. Supongamos que tenemos una clase llamada "Animal" que tiene un método llamado "hacerSonido". Ahora, creamos dos subclases de "Animal" llamadas "Perro" y "Gato". Cada una de estas subclases puede proporcionar su propia implementación del método "hacerSonido", que se comporta de manera diferente según la subclase.

Ejemplo en Java:
A continuación, se muestra un ejemplo en Java de cómo se puede implementar el polimorfismo utilizando la sobrecarga de métodos y la sobreescritura de métodos:

```
// Clase Animal
public class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido.");
    }
 }


 // Subclase Perro
```

```java
public class Perro extends Animal {
   public void hacerSonido() {
      System.out.println("El perro ladra.");
   }
   public void hacerSonido(String mensaje) {
      System.out.println("El perro dice: " + mensaje);
   }
}

// Subclase Gato
public class Gato extends Animal {
   public void hacerSonido() {
      System.out.println("El gato maulla.");
   }
}

// Clase principal
public class Principal {
   public static void main(String[] args) {
      Animal animal1 = new Perro();
      Animal animal2 = new Gato();
      animal1.hacerSonido();
      animal2.hacerSonido();
      Perro perro1 = new Perro();
      perro1.hacerSonido("¡Guau, guau!");
   }
}
```

En este ejemplo, la clase "Animal" es la superclase, y las clases "Perro" y "Gato" son las subclases. La subclase "Perro" tiene dos métodos llamados "hacerSonido": uno sin parámetros y otro con un parámetro de tipo String. La subclase "Gato" tiene su propia implementación del método "hacerSonido".

12. Polimorfismo en tiempo de compilación (sobrecarga) y polimorfismo en tiempo de ejecución (ligadura dinámica)

Existen dos tipos de polimorfismo: **polimorfismo en tiempo de compilación** (también conocido como **sobrecarga de métodos**) y **polimorfismo en tiempo de ejecución** (también conocido como **ligadura dinámica**).

- La **sobrecarga de métodos** es un tipo de polimorfismo en tiempo de compilación que permite definir múltiples métodos con el mismo nombre en una clase. Los métodos sobrecargados tienen diferentes tipos y/o número de parámetros, lo que significa que se comportan de manera diferente dependiendo de los argumentos que se les pasen.

Por ejemplo, supongamos que tenemos una clase llamada Calculadora con un método llamado sumar que acepta dos enteros como parámetros. Si queremos que la calculadora también pueda sumar dos números decimales, podemos definir otro método llamado sumar que acepte dos números decimales como parámetros:

```
public class Calculadora {
    public int sumar(int num1, int num2) {
        return num1 + num2;
    }

    public double sumar(double num1, double num2) {
        return num1 + num2;
    }
}
```

En este ejemplo, la clase Calculadora tiene dos métodos llamados sumar con la misma firma, pero diferentes tipos de parámetros. Cuando se llama al método sumar con dos enteros, se utilizará el primer método, y cuando se llama con dos números decimales, se utilizará el segundo método.

- La **ligadura dinámica** es un tipo de polimorfismo en tiempo de ejecución que permite que un objeto tome diferentes formas en tiempo de ejecución. En este caso, el comportamiento de un objeto se determina en tiempo de ejecución en lugar de tiempo de compilación.

Por ejemplo, supongamos que tenemos una clase llamada Animal con un método llamado hacerSonido. También tenemos dos subclases de Animal llamadas Perro y Gato que sobrescriben el método hacerSonido:

```
public class Animal {
    public void hacerSonido() {
        System.out.println("Haciendo sonido genérico");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("Guau");
    }
}

public class Gato extends Animal {
    public void hacerSonido() {
        System.out.println("Miau");
    }
}
```

En este ejemplo, la clase Animal tiene un método hacerSonido que simplemente imprime "Haciendo sonido genérico". Las subclases Perro y Gato sobrescriben el método hacerSonido y proporcionan su propia implementación. Ahora, si creamos un objeto de tipo Perro y llamamos al método hacerSonido, se imprimirá "Guau", mientras que si creamos un objeto de tipo Gato y llamamos al método hacerSonido, se imprimirá "Miau". En tiempo de ejecución, el objeto tomará la forma de la subclase correspondiente y se ejecutará el método correspondiente.

12.1 Polimorfismo: Comprobación estática y dinámica de tipos

El polimorfismo se refiere a la capacidad de objetos de diferentes clases para responder al mismo mensaje o método. Sin embargo, es importante distinguir entre la **comprobación estática y dinámica de tipos** en el contexto del polimorfismo.

La **comprobación estática de tipos** ocurre en tiempo de compilación y se utiliza para garantizar que los objetos sean compatibles con los métodos que se invocan en ellos. La **comprobación dinámica de tipos** ocurre en tiempo de ejecución y se utiliza para determinar el tipo real de un objeto en tiempo de ejecución.

En Java, la comprobación estática de tipos se realiza mediante la verificación del tipo de argumentos en tiempo de compilación. Por ejemplo, si se tiene una clase Animal y una clase Gato que hereda de Animal, y se llama a un método que toma un argumento de tipo Animal, se puede pasar un objeto de tipo Gato porque Gato es un subtipo de Animal.

```
class Animal {
    public void hacerSonido() {
        System.out.println("Haciendo sonidos como un animal");
    }
}

class Gato extends Animal {
    public void hacerSonido() {
        System.out.println("Miau");
    }
}

class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("Guau");
    }
}
```

```
class Main {
   public static void main(String[] args) {
      Animal miMascota = new Gato();
      miMascota.hacerSonido(); // La salida será "Miau"
   }
}
```

En este ejemplo, se crea un objeto de tipo Gato y se asigna a una variable de tipo Animal. Cuando se llama al método hacerSonido() en la variable miMascota, se llama al método correspondiente de la clase Gato porque se está haciendo uso del polimorfismo en tiempo de ejecución.

Sin embargo, la comprobación estática de tipos no siempre puede detectar todos los errores de tipos. Si se intenta llamar a un método que no existe en un objeto, se producirá un error en tiempo de compilación.

```
class Animal {
   public void hacerSonido() {
      System.out.println("Haciendo sonidos como un animal");
   }
}

class Gato extends Animal {
   public void hacerSonido() {
      System.out.println("Miau");
   }
}

class Perro extends Animal {
   public void hacerSonido() {
      System.out.println("Guau");
   }
```

```
}


class Main {
   public static void main(String[] args) {
      Animal miMascota = new Gato();
      miMascota.correr(); // Esto generará un error en tiempo de compilación
   }
}
```

En este ejemplo, se intenta llamar al método correr() en la variable miMascota, que es de tipo Animal. Sin embargo, la clase Animal no tiene un método llamado correr(), por lo que se producirá un error en tiempo de compilación.

La comprobación dinámica de tipos, por otro lado, se realiza en tiempo de ejecución y se utiliza para garantizar que un objeto sea del tipo correcto antes de realizar una operación en él.

13. Conversiones de tipos entre objetos (casting)

En programación el **casting** se refiere a la conversión de un tipo de dato a otro. En particular, en la programación orientada a objetos el casting se utiliza para convertir un objeto de una clase a otro objeto de una clase diferente.

Existen dos tipos de casting en Java: el **casting implícito** y el **casting explícito.**
El **casting implícito** se realiza de forma automática cuando se asigna un valor de un tipo de dato a una variable de un tipo diferente, siempre y cuando el valor pueda ser convertido sin pérdida de información.

Por ejemplo:

```
int a = 5;

double b = a; // casting implícito de int a double
```

En este caso, el valor de la variable a se convierte de forma implícita a un tipo double y se asigna a la variable b.

El **casting explícito**, por otro lado, se realiza de forma manual utilizando el operador de casting (tipo) antes del valor que se desea convertir. Por ejemplo:

```
int a = 5;

double b = a; // casting implícito de int a double
```

En este caso, se utiliza el casting explícito para convertir el valor de a de tipo double a tipo int, y se asigna el resultado a la variable b. Es importante tener en cuenta que al realizar un casting explícito se puede perder información, por lo que se debe tener cuidado al utilizarlo.

En la programación orientada a objetos, el casting se utiliza para convertir un objeto de una clase a otro objeto de una clase diferente. Por ejemplo:

```
public class Animal {

    public void hacerSonido() {

        System.out.println("Sonido de animal.");

    }

}
```

```
public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("Guau.");
    }
}

public class Gato extends Animal {
    public void hacerSonido() {
        System.out.println("Miau.");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Perro();
        a.hacerSonido(); // Imprime "Guau."

        Perro p = (Perro) a;
        p.hacerSonido(); // Imprime "Guau."

        Gato g = (Gato) a; // Error de compilación: Perro no se puede convertir
a Gato
    }
}
```

En este ejemplo, se crea una clase Animal y dos clases que heredan de ella, Perro y Gato. En la clase Main, se crea un objeto Perro y se almacena en una variable de tipo Animal. Después, se utiliza el casting explícito para convertir el objeto Animal a un objeto Perro. Esto es posible porque Perro es una subclase de Animal. Se llama a hacerSonido() en ambas variables y en ambos casos se imprime "Guau."
Sin embargo, al intentar realizar un casting a la clase Gato, se produce un error de compilación porque Perro no se puede convertir a Gato.

Importante

Es importante tener en cuenta que el casting entre clases solo es posible si hay una relación de herencia entre ellas.

Existen dos **tipos de casting**:

13.1 Upcasting

Es cuando se convierte un objeto de una subclase a su superclase correspondiente. Este tipo de conversión se realiza automáticamente por el compilador, ya que no hay riesgo de pérdida de información.

Ejemplo:
Supongamos que tenemos la siguiente jerarquía de clases:

```
public class Animal {
    public void hacerSonido() {
      System.out.println("Sonido del animal");
    }
  }

  public class Perro extends Animal {
    public void hacerSonido() {
      System.out.println("Guau guau");
    }
  }

  public class Gato extends Animal {
    public void hacerSonido() {
      System.out.println("Miau miau");
    }
  }
```

Luego, podemos hacer lo siguiente:

```
Animal animal1 = new Perro();
Animal animal2 = new Gato();


animal1.hacerSonido(); // Output: Guau guau
animal2.hacerSonido(); // Output: Miau miau
```

En este caso, estamos creando un objeto de la subclase Perro y asignándolo a una variable de tipo Animal (que es su superclase). Lo mismo sucede con el objeto de la subclase Gato. Luego, al llamar al método hacerSonido() de cada objeto, se ejecuta el método correspondiente de la subclase (y no el de la superclase).

13.2 Downcasting

Es cuando se convierte un objeto de una superclase a una subclase correspondiente. En este caso, es necesario hacer una conversión explícita utilizando el operador (tipoSubclase).
Ejemplo:
Supongamos que tenemos la siguiente jerarquía de clases:

```
public class Animal {
    public void hacerSonido() {
      System.out.println("Sonido del animal");
    }
  }

  public class Perro extends Animal {
    public void hacerSonido() {
      System.out.println("Guau guau");
    }

    public void ladrar() {
      System.out.println("¡Estoy ladrando!");
```

```
    }
  }

  public class Gato extends Animal {
    public void hacerSonido() {
      System.out.println("Miau miau");
    }

    public void maullar() {
      System.out.println("¡Estoy maullando!");
    }
  }
```

Luego, podemos hacer lo siguiente:

```
Animal animal = new Perro();
Perro perro = (Perro) animal;

animal.hacerSonido(); // Output: Guau guau
perro.ladrar(); // Output: ¡Estoy ladrando!
```

En este caso, estamos creando un objeto de la subclase Perro y asignándolo a una variable de tipo Animal (que es su superclase). Luego, hacemos un downcasting para convertir el objeto de tipo Animal en un objeto de tipo Perro. Luego, al llamar al método hacerSonido() de la variable animal, se ejecuta el método correspondiente de la subclase Perro. Además, podemos llamar al método ladrar() que está definido solamente en la subclase Perro.

13.3. Diferencia entre casting implícito y el casting explícito vs Upcasting y Downcasting

La principal diferencia entre los cuatro tipos de casting es la **dirección de la conversión** y si es necesaria una sintaxis explícita para realizar la conversión.

- En el **casting implícito** la conversión se realiza automáticamente por el compilador sin necesidad de especificarla en el código. Por ejemplo,

si se asigna un valor de tipo **int** a una variable de tipo **double**, el compilador realiza una conversión implícita de tipo para ajustar el valor a un número de punto flotante.

- En el **casting explícito** se utiliza una sintaxis explícita para indicar que se quiere realizar la conversión. Por ejemplo, si se quiere convertir un valor de tipo **double** a un tipo **int**, se puede utilizar la sintaxis (int) **doubleValue** para realizar el casting explícito.

- El **upcasting** se refiere a la conversión de un objeto de una subclase a su superclase correspondiente. En este caso, la conversión es segura y se realiza automáticamente, ya que el objeto de la subclase ya tiene todos los atributos y métodos de la superclase. Por ejemplo, si se tiene una clase Animal y una clase Perro que hereda de Animal, se puede realizar upcasting al convertir un objeto de la clase Perro en un objeto de la clase Animal.

- El **downcasting** se refiere a la conversión de un objeto de una superclase a una subclase correspondiente. En este caso, la conversión no es segura, ya que el objeto de la superclase puede no tener todos los atributos y métodos de la subclase. Por lo tanto, se necesita una sintaxis explícita para realizar el downcasting.
  Por ejemplo, si se tiene un objeto de la clase Animal y se quiere convertirlo en un objeto de la clase Perro, se puede utilizar la sintaxis (Perro) animalObjeto para realizar el downcasting. Si el objeto de la superclase no es realmente un objeto de la subclase, se producirá una excepción en tiempo de ejecución.

14. Clases y tipos genéricos o parametrizados

14.1 Introducción a las clases y tipos genéricos o parametrizados

Las **clases y tipos genéricos o parametrizados** son una característica avanzada de programación que permite crear clases y métodos que pueden trabajar con diferentes tipos de datos sin tener que reescribir el código para cada tipo. En lugar de definir una clase o método para cada tipo de datos, los tipos genéricos permiten definir una sola clase o método que puede manejar diferentes tipos de datos en tiempo de compilación.

14.2 ¿Qué son las clases genéricas?

Las **clases genéricas** son clases que pueden trabajar con cualquier tipo de datos. En lugar de especificar un tipo de datos concreto se utiliza un tipo de parámetro en su lugar. Por ejemplo, en lugar de crear una clase que solo pueda trabajar con cadenas de texto, se puede crear una clase que pueda trabajar con cualquier tipo de objeto. Para crear una clase genérica, se utiliza la sintaxis <T>, donde T es el tipo de parámetro.

```
public class MiClase<T> {
    private T objeto;

    public MiClase(T objeto) {
        this.objeto = objeto;
    }

    public T getObjeto() {
        return objeto;
    }
}
```

En este ejemplo, T es un tipo de parámetro que se utilizará para definir el tipo de objeto que se utilizará en la clase. La clase MiClase tiene un campo privado llamado objeto que es del tipo de parámetro T. El constructor de la clase toma

un objeto del tipo T y lo almacena en el campo objeto. La clase también tiene un método público getObjeto() que devuelve el objeto almacenado en el campo objeto.

14.3 ¿Qué son los métodos genéricos?

Los **métodos genéricos** son métodos que pueden trabajar con cualquier tipo de datos. Al igual que las clases genéricas, se utiliza un tipo de parámetro en lugar de un tipo de datos concreto. Los métodos genéricos se definen utilizando la sintaxis <T> antes del tipo de retorno del método.

```
public static <T> T devuelvePrimerElemento(List<T> lista) {
    if (!lista.isEmpty()) {
        return lista.get(0);
    }
    return null;
}
```

En este ejemplo, T es un tipo de parámetro que se utilizará para definir el tipo de objeto que se utilizará en el método. El método devuelvePrimerElemento toma una lista de objetos del tipo T y devuelve el primer objeto de la lista. Si la lista está vacía, el método devuelve null.

14.4 ¿Qué son los tipos comodín?

Los **tipos comodín** son una característica de los tipos genéricos que permiten trabajar con diferentes tipos de objetos en una misma estructura de datos. Se utilizan para permitir que una clase o método pueda trabajar con diferentes tipos de objetos, siempre y cuando sean subtipos de un tipo común.
Existen dos **tipos comodín**: '<?>' y '<? extends Tipo>'.

- **<?>**: Se utiliza para indicar que se acepta cualquier tipo de objeto.
- **<? extends Tipo>**: Se utiliza para indicar que se acepta cualquier tipo que sea

15. Resumen

Como hemos visto en la unidad, has aprendido que la programación orientada a objetos (POO) se basa en el concepto de objetos. Estos objetos son instancias de clases, y has interactuado con ellos a través de métodos y atributos. Asimismo, has visto las clases, que son plantillas que han definido los atributos y métodos que tienen los objetos que has creado a partir de ellas. En relación a esto, has aprendido que los objetos son copias de la clase. Importante destacar que éstos han tenido valores específicos para sus atributos. Por otro lado, los atributos han sido las características que han tenido los objetos, mientras que los métodos han sido las acciones que han podido realizar.

En cuanto a la relación entre clases, has entendido que la herencia juega un papel importante. En este sentido, has visto que la herencia es una relación entre dos clases, donde una subclase ha heredado atributos y métodos de una superclase. De igual manera, la sobrecarga de métodos ha permitido definir varios métodos con el mismo nombre, pero con diferentes parámetros.

Además, has aprendido que el polimorfismo ha sido la capacidad de un objeto de ser referenciado por múltiples tipos de datos, así como otro concepto relevante, que ha sido la ligadura dinámica, la cual ha implicado la selección del método a llamar en tiempo de ejecución en lugar de tiempo de compilación.

Por otro lado, has aprendido que las interfaces se tratan de clases abstractas que no han tenido una implementación definida para sus métodos. Por lo tanto, cualquier clase que haya implementado una interfaz ha debido proveer su propia implementación para todos los métodos que se han definido en la interfaz.

En relación a la flexibilidad en el manejo de tipos de datos, has conocido que los genéricos han permitido crear clases y métodos que han podido trabajar con diferentes tipos de datos. Además, has visto que las excepciones han sido errores que han podido ocurrir durante la ejecución del programa y que han podido ser manejados por el código. Finalmente, has comprendido que los paquetes han sido agrupaciones de clases y otros paquetes que se han utilizado para organizar y modularizar el código

Glosario de términos

- **Atributo**: una variable que se encuentra dentro de una clase y describe una propiedad del objeto.
- **Casting**: la conversión de un tipo de dato a otro.
- **Clase**: una plantilla para crear objetos que define sus atributos y comportamientos.
- **Constructor**: método especial que se ejecuta automáticamente al crear un objeto de una clase.
- **Herencia**: relación entre dos clases en la que una subclase adquiere atributos y métodos de una superclase.
- **Interfaces**: plantilla que define un conjunto de métodos que una clase debe implementar.
- **Ligadura dinámica**: selección del método a ejecutar en tiempo de ejecución en lugar de tiempo de compilación.
- **Ligadura estática**: selección del método a ejecutar en tiempo de compilación en lugar de tiempo de ejecución.
- **Método**: una función definida dentro de una clase que puede ser llamada en un objeto.
- **Objeto**: una instancia de una clase.
- **Polimorfismo**: la capacidad de un objeto de ser referenciado por múltiples tipos de datos.
- **Sobreescritura de métodos**: mecanismo por el cual una subclase puede proporcionar su propia implementación para un método ya definido en su superclase.
- **Subclase**: una clase que hereda de otra clase.
- **Superclase**: clase que hereda de otra clase.

Bibliografía

Schildt, H. (2018). Java: A beginner's guide (7th ed.). McGraw-Hill Education.

Bloch, J. (2017). Effective Java (3rd ed.). Addison-Wesley Professional.

Tech Hub <FP> euroformac